



Contenido

01

¿Qué son las estructuras de Control ?

02

Base de las condicionales en Java

03

La estructura if

04

La estructura if - else

05

La estructura if – else al detalle

06

Estructura switch

07

Else if vs switch

08

Operador Ternario

09

Caso práctico combinado

10

Errores Comunes

11

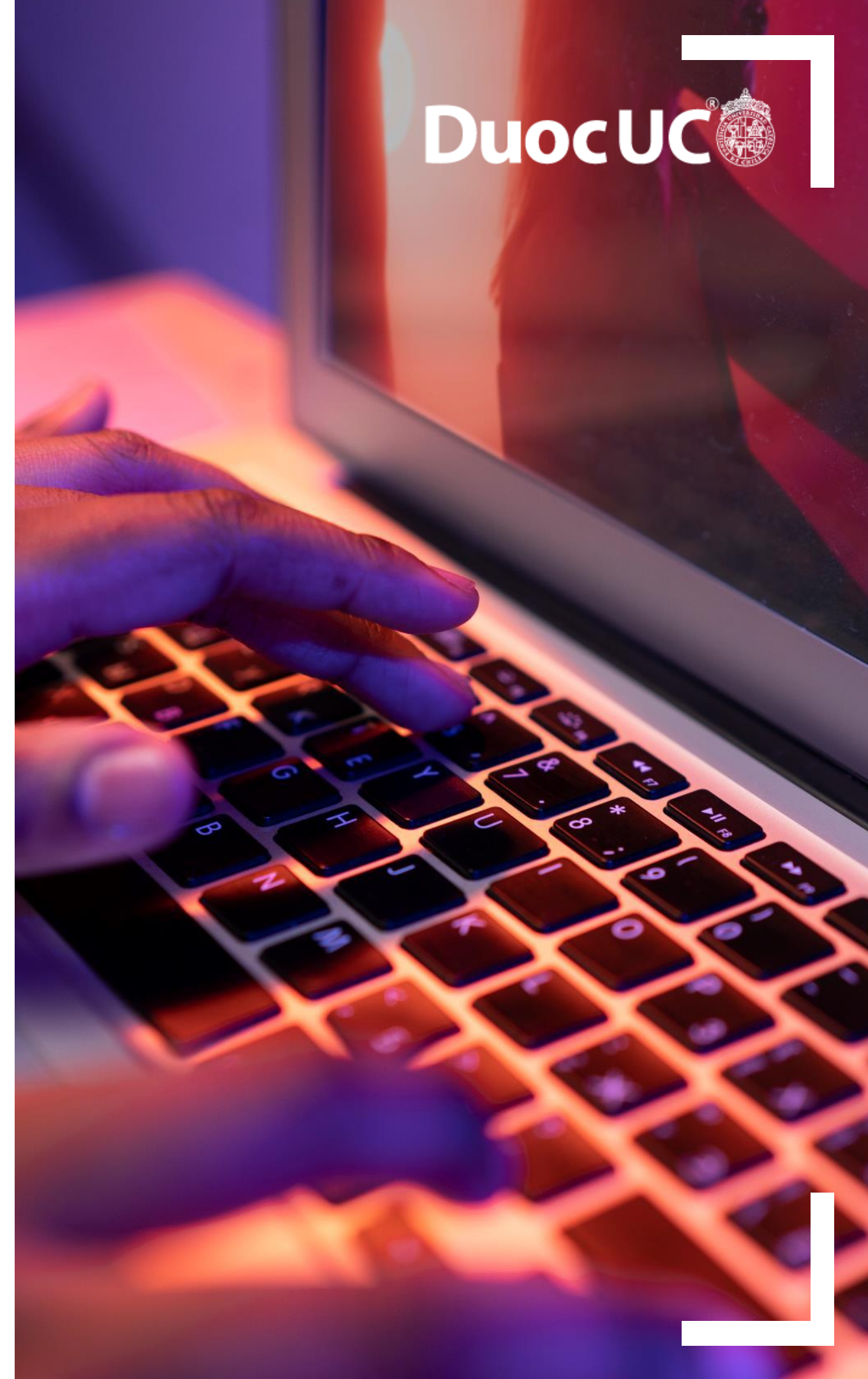
Buenas prácticas en Java

12

Resumen visual estructuras de control

13

Conclusiones y Próximos pasos



</> Fundamentos del Lenguaje

Java

Estructuras de Control en Java

if, else, else if, switch y operador ternario.



Condicionales
if, else if, switch



Evaluaciones
edad >= 18, opcion

```
ControlFlow.java

// 1. Condicional if-else
if (edad ≥ 18) {
    estado = "Mayor";
} else {
    estado = "Menor";
}

// 2. Operador Ternario
estado = (edad ≥ 18) ? "Mayor" : "Menor";

// 3. Estructura Switch
switch (opcion) {
    case 1:
        iniciar();
        break;
    case 2:
        salir();
        break;
}
```



¿Qué son las Estructuras de Control?

Dirigiendo la ejecución del programa

Definición

Las **estructuras de control** son instrucciones que permiten modificar el flujo de ejecución secuencial de un programa, evaluando condiciones para decidir qué código ejecutar.



Toman decisiones

El programa "piensa" y elige un camino según la situación.



Controlan el flujo

Rompen la ejecución línea por línea, saltando código si es necesario.



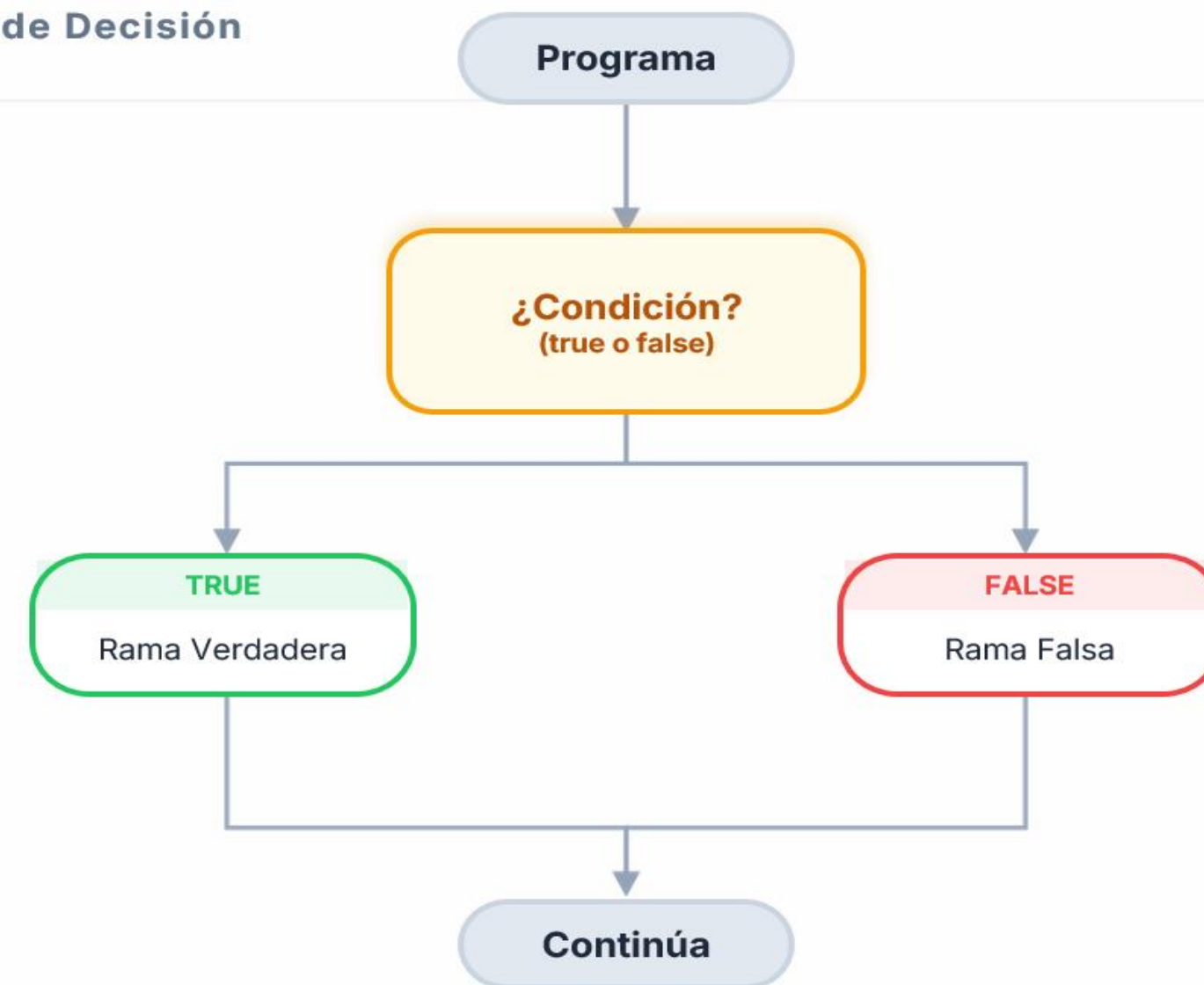
Usan booleanos

Dependen de expresiones que siempre resultan en true o false.

Concepto Clave:

i Sin ellas, un programa solo haría lo mismo una y otra vez.

Flujo de Decisión



Base de las Condiciones en Java

Las estructuras de control dependen de expresiones que se evalúan como verdaderas o falsas para decidir qué camino tomar.

Tipo Boolean

Es la base del control de flujo. Toda condición en Java debe resolverse obligatoriamente a un valor `true` o `false`.

Comparaciones

Operadores como `>`, `<`, `=` y `≠` evalúan la relación entre valores y devuelven un resultado booleano.

Operadores Lógicos

Permiten combinar o invertir condiciones usando `&&` (AND), `||` (OR) y `!` (NOT) para crear reglas complejas.

Evaluación de Expresiones Booleanas

Comparación Simple

`edad ≥ 18`

`true`

Condición Lógica Múltiple

`nota ≥ 60 && asistencia ≥ 80`

`false`

Negación Lógica

`!error`

`true`



La estructura if

Ejecutar código solo si se cumple una condición

¿Cómo funciona?

La sentencia `if` (si...) evalúa una condición booleana. Si el resultado es `true`, ejecuta el bloque de código asociado; si es `false`, lo salta.

?

Condición

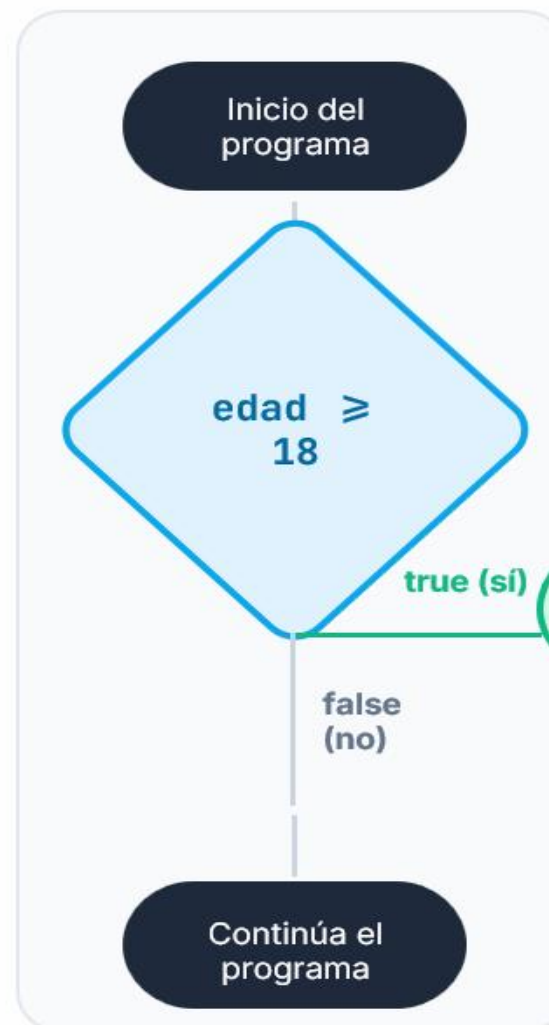
Debe ir entre paréntesis y devolver `true` o `false`.

</>

Bloque de Código

Limitado por llaves `{ }`. Contiene las instrucciones a ejecutar.

Flujo de Decisión



Ejemplo.java

```
// Variable inicial
int edad = 20;
// Evaluamos la condición
if (edad ≥ 18) {
// Este bloque se ejecuta
    System.out.println("Puede entrar");
}
```

```
System.out.println("Fin");
```



La estructura if-else

Dos caminos posibles según una condición

Bifurcación del Flujo

Si la condición del **if** es falsa, el programa ejecuta automáticamente el bloque **else**. Solo se sigue uno de los dos caminos.



Bloque if (Verdadero)

Se ejecuta si la condición se cumple.



Bloque else (Falso)

La alternativa si la condición no se cumple.



Manejo de Casos por Defecto

Asegura que el programa siempre tenga una respuesta.

Ejemplo Visual: Toma de Decisión



Compra.java

```
double saldo = 50.0;
double precio = 75.0;

if (saldo ≥ precio) {
    System.out.println("Compra exitosa");
    saldo = saldo - precio;
} else {
    System.out.println("Saldo insuficiente...");
}
```





La Estructura else if

Evaluando múltiples condiciones en cascada



Decisiones Múltiples

Usa `else if` para evaluar **varias condiciones en orden**. El programa verifica cada una hasta encontrar la primera que sea verdadera.

1. Orden de Evaluación

Se evalúa de arriba hacia abajo. Si una condición es `true`, se ejecuta su bloque y se ignora el resto.

2. Exclusividad

Solo se ejecutará **un** bloque de código, incluso si múltiples condiciones pudieran ser verdaderas (gana la primera).

3. El `else` Final

Opcional, pero recomendado. Actúa como el caso "por defecto" si ninguna condición anterior se cumple.

Escalera de Condiciones: Evaluación de Notas

`if (nota ≥ 90)`

★ **Excelente**

Se ejecuta si es 90 o más. Fin.

`else if (nota ≥ 70)`

✓ **Aprobado**

Si no fue ≥ 90, pero sí ≥ 70.

`else if (nota ≥ 60)`

⚠ **Recuperación**

Si no fue ≥ 70, pero sí ≥ 60.

`else`

✗ **Reprobado**

Si ninguna de las anteriores se cumplió.



Estructura Switch

Múltiples casos definidos para una sola variable

¿Cuándo usar switch?

Es ideal cuando una única variable puede tomar varios valores exactos y conocidos (ej. opciones de menú, días, estados).



Casos Específicos (case)

Define cada valor posible de la variable evaluada.



Ruptura (break)

Obligatorio al final de cada caso para salir del switch y no ejecutar los siguientes.



Por Defecto (default)

Se ejecuta si ningún case coincide (similar al else final).

Tipos permitidos:

Funciona con `int`, `char`, `String`, `enum` y tipos envoltentes.

Flujo del Switch: Menú de Opciones

```
switch (opcion) {
```

```
case 1:    iniciarJuego();
```

```
break;
```

```
case 2:    guardarPartida();
```

```
break;
```

```
case 3:    salir();
```

```
break;
```

```
default:   mostrarError("Inválido");
```

```
// opcional
```

```
}
```





else if vs switch

¿Cuándo usar cada estructura de decisión múltiple?

? ¿Cómo Elegir?

Ambas estructuras permiten manejar múltiples caminos, pero están optimizadas para **diferentes escenarios** de evaluación.



Usa 'else if' cuando:

Evalúas rangos (ej. > 10 y < 20), condiciones lógicas complejas, o tipos no soportados por switch.



Usa 'switch' cuando:

Tienes una sola variable y quieres compararla contra valores exactos (int, String, Enum).



Rendimiento y Lectura

Switch suele ser más rápido y limpio al leer si hay más de 3 casos exactos.

Criterio rápido: ¿Es un rango? `else if`. ¿Es un menú? `switch`.

Comparación de Estructuras



else if

Ideal para rangos y lógicas complejas.

```
if (nota ≥ 90) {  
  print("A");  
} else if (nota ≥ 80) {  
  print("B");  
} else {  
  print("C");  
}
```

Evalúa expresiones booleanas.

Permite múltiples variables.

Puede ser lento con muchos casos.



switch

Ideal para valores exactos específicos.

```
switch (dia) {  
  case1:  
    print("Lun");  
    break;  
  case2:  
    print("Mar");  
    break;  
}
```

Evalúa un solo valor (igualdad).

Tipos limitados (int, String...).

Más legible para menús.



Operador Ternario

La alternativa concisa al bloque if-else para asignaciones

Forma Breve

El operador ternario (`? :`) permite evaluar una condición y devolver un valor en una sola línea.

-  **Sintaxis Corta:** Estructura de 3 partes: condición, valor si es verdadero y valor si es falso.
-  **Retorna un Valor:** A diferencia de un if tradicional, siempre devuelve un resultado para asignar.
-  **Código Limpio:** Ideal para asignaciones simples, evitando abrir y cerrar múltiples llaves.

Cuándo Sí conviene:

- ✔ Para asignaciones directas. Evita anidarlos para mantener la legibilidad.

Estructura y Ejemplo Visual

Anatomía del Operador

```
variable = condición ? valor_si_true : valor_si_false ;
```

¿Se cumple?

Si es Sí (true)

Si es NO (false)

Ejemplo en Java

// En lugar de usar 4 líneas de if-else:

```
String msj = (edad ≥ 18) ? "Mayor" : "Menor";
```

```
System.out.println(msj);
```



Caso Práctico Combinado

Integrando if, else if, switch y operador ternario

Sistema Escolar

En un programa real, las estructuras de control conviven. Vemos un sistema que valida un alumno, procesa su opción de menú y evalúa su nota final.



1. Validación (if)

Verifica si el estudiante está activo.



2. Menú (switch)

Elige la acción: ver notas, pagar, salir.



3. Evaluación (else if / ?:)

Calcula la calificación y el estado final.

Flujo Completo

```
boolean activo = true;
int opcion = 1;
int nota = 85;

// 1. Validación Principal
if (!activo) {
    System.out.println("Usuario inactivo");
} else {
    // 2. Procesar Menú
    switch (opcion) {
        case 1:
            System.out.println("--- Notas ---");
            // 3. Evaluación Detallada
            if (nota >= 90) System.out.println("A"); ← Sin Llaves de apertura y clausura
            else if (nota >= 80) System.out.println("B");
            else System.out.println("F");

            // 4. Operador Ternario para estado rápido
            String estado = (nota >= 60) ? "Aprobado" : "Reprobado";
            System.out.println("Estado: " + estado);
            break;
        case 2:
            System.out.println("Pagos");
            break;
        default:
            System.out.println("Opción inválida");
    }
}
```



⚠ Errores Comunes

Identifica y soluciona problemas frecuentes al usar estructuras de control

🔪 ¿Qué suele salir mal?

❏ Olvidar las llaves {}

Sin llaves, un if solo aplica a la primera línea siguiente.

= Confundir = con ==

Intentar asignar en lugar de comparar dentro de la condición.

➤ Omitir break en switch

El flujo caerá en cascada ejecutando los casos de abajo.

? Abusar del ternario

Anidar operadores ternarios vuelve el código incomprensible.

❏ Mal orden en else if

Poner una condición general antes que una específica bloquea el flujo.

🔧 Error Consecuencia Solución

Bloques sin llaves

```
if (false)
  accionA();
  accionB();
```

→ accionB() se ejecuta SIEMPRE

```
if (false) { accionA(); accionB(); }
```

Fall-through en switch

```
case 1: print("A");
case 2: print("B");
```

→ Imprime "A" y luego "B"

```
case 1: print("A"); break;
```

Orden Lógico en if

```
if (nota > 60) ...
else if (nota > 90) ...
```

→ Nunca entrará al > 90

```
if (nota > 90) ... else if (nota > 60) ...
```

✓ De más estricto a menos



✓ Buenas Prácticas

Consejos para escribir estructuras de control limpias y legibles

💡 Reglas de Oro

🔹 Condiciones Claras

Evita negaciones dobles o bloques lógicos muy largos. Extrae la lógica a variables descriptivas.

🔸 Simplificar Ternarios

Úsalos solo para asignaciones cortas y directas. Nunca anides operadores ternarios.

☰ Preferir Switch

Si evalúas valores exactos (ej. menús o estados fijos), usa switch en vez de muchos else if.

💬 Comentarios y Paréntesis

Agrupar expresiones con () y documenta la lógica de negocio compleja para mayor claridad.

📋 Checklist Visual: Lo que SÍ debes hacer



Evitar anidamiento excesivo (Guard Clauses)

```
if (error) return; // Salida temprana
```



Mantener el Ternario Corto

```
String msg = esValido ? "Sí" : "No";
```



Siempre incluir 'default' en un Switch

```
default: System.out.println("Opción inválida");
```



Comparar Strings con .equals() en los 'if'

```
if (rol.equals("Admin")) { ... }
```



Resumen Visual: Estructuras de Control

Hoja de repaso rápido de if, else if, switch y operador ternario



Condiciones Básicas

if else if else

Basado en evaluaciones **booleanas** (true/false).
Ideal para rangos de valores y lógica de validación compleja.

```
if (nota ≥ 90) {  
    System.out.println("A");  
} else if (nota ≥ 60) {  
    System.out.println("B");  
} else {  
    System.out.println("C");  
}
```

- ✓ Evalúa condiciones en orden
- ✓ Ejecuta solo la primera que cumple



Casos Múltiples

switch case break default

Evalúa una variable contra múltiples **valores exactos**. Más limpio que muchos else if consecutivos.

```
switch (opcion) {  
    case 1:  
        guardar();  
        break;  
    default:  
        salir();  
}
```

- i No olvides el **break**
- i Excelente para menús u opciones fijas



Asignación Rápida

? : (Ternario)

Alternativa compacta a if/else para asignar un valor en una sola línea. Útil para condiciones simples.

```
// Condición ? si_cumple : si_falla  
  
String estado = (edad ≥ 18)  
? "Mayor de edad"  
: "Menor de edad";
```

"No lo uses para lógica compleja o múltiples condiciones anidadas; la claridad siempre es más importante que ahorrar líneas."



Conclusiones y Próximos Pasos

Cierre de Estructuras de Control

🔑 Ideas Clave

Comprender las **estructuras de control** es fundamental para crear programas que tomen decisiones correctas en Java.



La condición define la decisión

if/else maneja caminos lógicos basados en expresiones booleanas.



Switch organiza casos exactos

Ideal para múltiples valores específicos, evitando cadenas largas de else if.



El ternario simplifica

Asignaciones cortas y legibles en una sola línea.

✓ **Resultado: Código más claro, predecible y mantenible**

Estructura Fundamental



→ Próximos Pasos

1 Bucles (for/while)

2 Métodos y Funciones

3 Validaciones Complejas



Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación